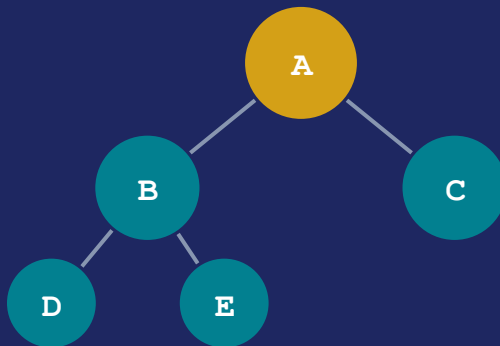


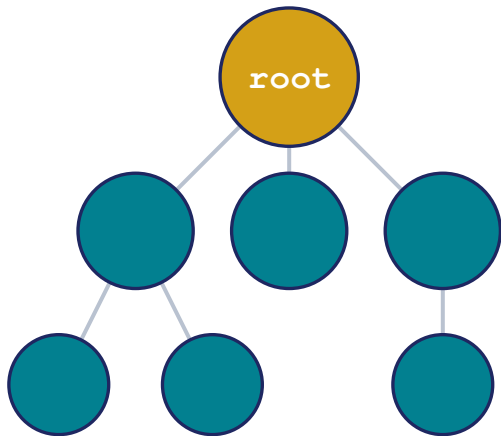
Trees: Beyond the Straight Line

Tree Terminology, Binary Trees and Applications

Unit III • Lecture 21 (first lecture of Unit III)



What is a tree?



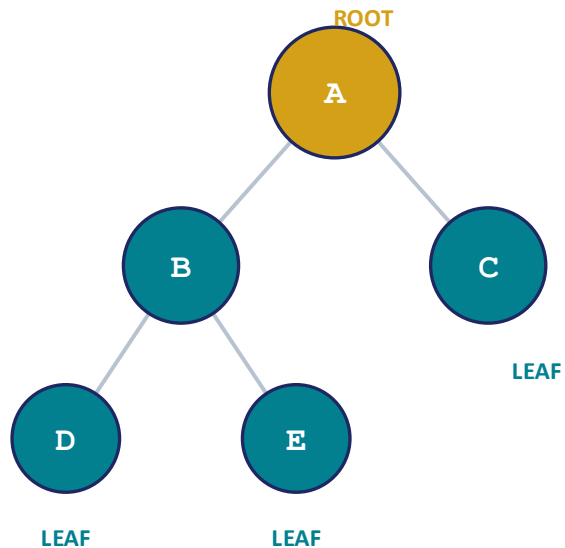
A tree

Definition

A tree is a set of nodes connected by edges such that: there is exactly ONE root, and every other node has exactly ONE parent — no cycles, everything reachable.

- Contrast with Units I-II: every structure so far was LINEAR — one thing led to at most one next thing.
- A tree lets one node lead to SEVERAL next things — it branches.
- A linked list is technically a tree where every node has at most one child — the most boring possible tree.

Core vocabulary, on one small tree



Root: The one node with no parent — A here.

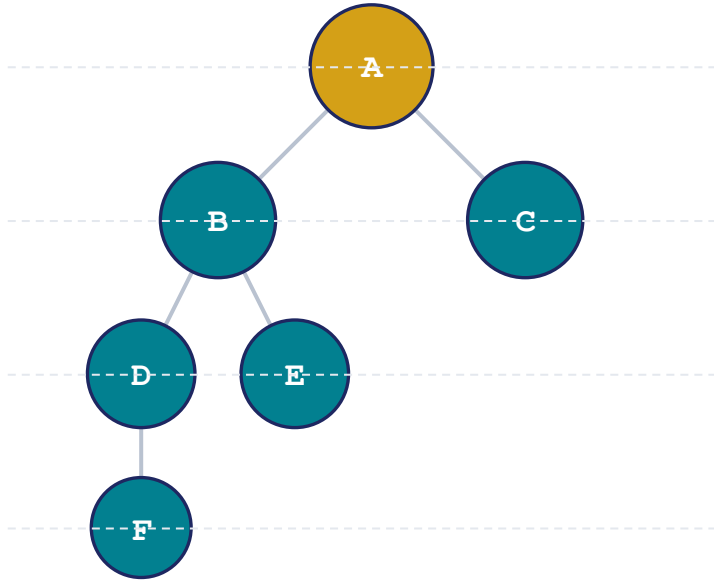
Parent / Child: B and C are children of A; A is their parent.

Siblings: B and C share the same parent — they are siblings.

Leaf: A node with NO children — C, D, E here.

Edge: The connection between a parent and a child.

Height, depth, and level: the same tree, measured

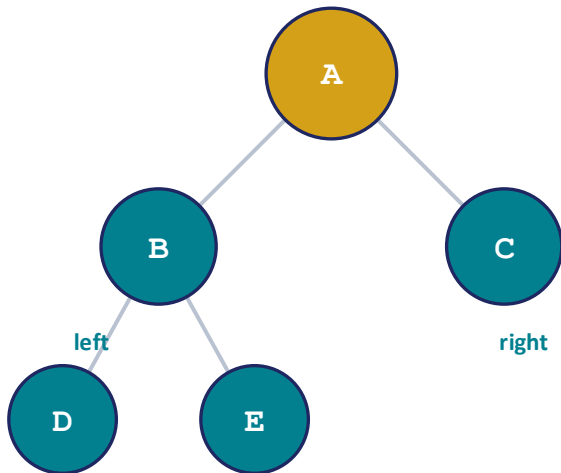


Level: how many edges from the ROOT (A is level 0, F is level 3).

Depth of a node: same idea, its own distance from the root.

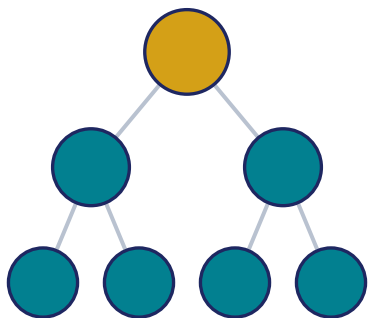
Height of the TREE: the depth of its deepest leaf — here, height = 3 (path A-B-D-F).

Binary tree: at most TWO children, left and right



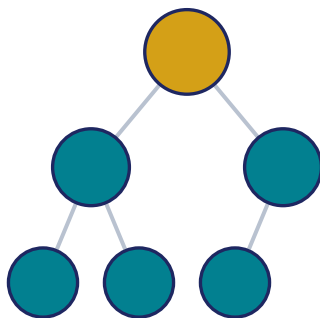
- Every node has at most 2 children — conventionally LEFT and RIGHT, and the two are DIFFERENT positions (a node with only a right child is not the same shape as one with only a left child).
- A general tree allows any number of children per node; binary is a specific, stricter case.
- "Binary" is doing real work in that name — not just "a small tree."

Shapes worth naming: full, complete, skewed



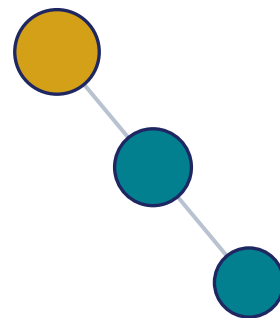
FULL

every node has 0 or 2 children



COMPLETE

every level full, except
the last, filled left to right

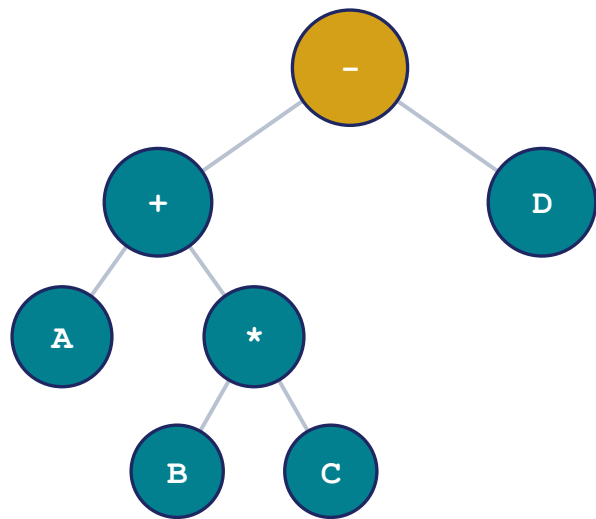


SKEWED

every node has only
one child — a disguised list

Why this matters: Lecture 22's array representation is efficient for COMPLETE trees and wasteful for SKEWED ones — shape drives implementation choice.

Application: expression trees (a Lecture 9 callback)



for $A + B * C - D$

Preorder = prefix

$- + A * B C D$

Inorder = infix

$A + B * C - D$

Postorder = postfix

$A B C * + D -$

Postorder output *EXACTLY* matches Lecture 9's worked answer for this expression — postfix was never a separate trick, just one walk of this tree.

Trees are everywhere hierarchy matters

File systems

Folders contain files and other folders — a tree, root is the drive/root directory.

Decision trees

Each internal node is a yes/no question; each leaf is a final decision.

Huffman coding (L26)

Builds a tree bottom-up from character frequencies to create optimal codes.

Binary search trees (L24)

Keeps data ordered so searching is faster than a linear scan.

The vocabulary for everything ahead

- A tree: one root, every other node has exactly one parent, no cycles. A linked list is the most boring possible tree.
- Vocabulary that will not be re-explained: root, parent/child, sibling, leaf, edge, height, depth, level, subtree.
- Binary tree: at most 2 children, left and right are distinct positions — not just "any small tree."
- Shape matters: full, complete, and skewed trees behave very differently once Lecture 22 picks a representation.
- Expression trees: preorder/inorder/postorder ARE prefix/infix/postfix — Lecture 9's postfix was always just one tree walk.

EXIT TICKET Draw the expression tree for $A * (B + C) - D$. Give its preorder, inorder, and postorder traversals.